



Guía 07 - Control de Versiones con Git y GitHub

1. TÍTULO: Guía 07 - Control de Versiones con Git y GitHub

2. DATOS INFORMATIVOS

- **Asignatura** : Desarrollo de Software I
- **Semestre Académico** : 2026-1
- **Ciclo** : VI
- **Categoría** : Obligatorios de Estudios Específicos (OEEs)
- **Docente** : Mtro. Ing. Luis Alvaro Monzón Condori
- **Duración** : 04 horas

3. COMPETENCIAS A SEGUIR.

Al final la guía, el estudiante será capaz de:

- Gestionar versiones de un proyecto web utilizando Git.
- Aplicar commits para registrar cambios en el desarrollo del sistema.
- Integrar repositorios locales con GitHub.
- Utilizar ramas para organizar el desarrollo de funcionalidades.
- Comprender el rol del control de versiones en el ciclo de desarrollo de software.

4. REQUISITOS.

- Visual Studio Code.
- Git instalado.
- Cuenta en GitHub.
- Proyecto web básico (HTML, CSS, JavaScript).

5. BASES TEÓRICAS.

5.1. ¿Qué es el Control de Versiones?

El control de versiones es un sistema que permite **registrar, gestionar y recuperar cambios en los archivos** de un proyecto a lo largo del tiempo.

En el desarrollo de software, esto permite:

- Evitar pérdida de código
- Recuperar versiones anteriores
- Comparar cambios realizados
- Mantener un historial del proyecto

Ejemplo práctico:

Un estudiante desarrolla una página web:

Día 1: versión básica

Día 2: agrega estilos

Día 3: agrega validaciones

Con control de versiones, cada cambio queda registrado y se puede volver atrás si algo falla

5.2. Importancia del Control de Versiones en el Desarrollo de Software

El control de versiones es fundamental porque:

- El desarrollo es progresivo.
- Se realizan múltiples modificaciones.
- Se trabaja con proyectos reales.
- Se requiere orden y trazabilidad.

Sin Control de Versiones:

- Se pierde código fácilmente.
- No se sabe qué cambió.



- No se puede trabajar en equipo.

Con Git:

- Se tiene control total del sistema.
- Se puede evolucionar el software de forma ordenada.

5.3. ¿Qué es Git?

Git es un sistema de **Control de Versiones Distribuido** que permite gestionar cambios en un proyecto de manera eficiente.

Características principales:

- Guarda historial completo del proyecto.
- Funciona de manera local (sin internet).
- Permite múltiples versiones del sistema.
- Soporta trabajo colaborativo.

Concepto clave:

- Git no guarda archivos...
- Guarda estados del proyecto (snapshots)

5.4. Estructura de Trabajo en Git

Git trabaja con tres áreas principales:

- **Working Directory:** Es donde el desarrollador trabaja directamente (archivos editados).
- **Staging Area:** Zona donde se preparan los cambios antes de guardarlos.
- **Repository:** Donde se almacenan los cambios confirmados (commits).

5.5. ¿Qué es un commit?

Un commit es un registro de cambios en el proyecto.

Cada commit incluye:

- Cambios realizados.
- Autor del cambio.
- Fecha.
- Mensaje descriptivo.

Buenas prácticas:

- Mensajes claros
- Cambios pequeños y organizados

5.6. Repositorio Local y Repositorio Remoto

- **Repositorio local:** Se encuentra en la computadora del desarrollador.
- **Repositorio remoto:** Se encuentra en plataformas como GitHub.

Relación:

- **Local** → Desarrollo
- **Remoto** → Respaldo + Colaboración

5.7. ¿Qué es GitHub?

GitHub es una plataforma que permite:

- Almacenar repositorios en la nube.
- Compartir código.
- Trabajar en equipo.
- Gestionar versiones de forma remota.



5.8. Ramas (Branches)

Las ramas permiten trabajar en diferentes versiones del proyecto sin afectar la versión principal.

Ejemplo:

- **main** → versión estable
- **feature-login** → nueva funcionalidad

Ventajas:

- Desarrollo paralelo.
- Evita errores en producción.
- Facilita pruebas.

5.9. Flujo de trabajo básico con Git

Modificar archivo

- `git add`
- `git commit`
- `git push`

Interpretación:

1. Se modifica el sistema.
2. Se prepara el cambio.
3. Se guarda el cambio.
4. Se sube a GitHub.

5.10. Relación con el Desarrollo de Software

Git es una herramienta clave en:

- Desarrollo Frontend.
- Desarrollo Backend.
- Integración continua.
- Trabajo en equipo.

En este curso, se utiliza para:

- Controlar el proyecto web.
- Documentar avances.
- Preparar al estudiante para entornos reales.

6. DESARROLLO DE LA GUÍA.

6.1. ACTIVIDAD 01: Instalación y Control de Versiones Local.

Objetivo: Instalar, configurar Git y registrar los primeros cambios de un proyecto web utilizando control de versiones.

Contexto: El estudiante desarrollará un sistema web básico y registrará su evolución mediante commits.

PARTE A: Instalación de Git

Paso 01: Descargar Git

- Ingresar a la página oficial: <https://git-scm.com>
- Descargar la versión correspondiente a su sistema operativo (Windows).

Paso 02: Instalar Git

Durante la instalación:

- Mantener opciones por defecto
- Hacer clic en Next hasta finalizar



Paso 03: Verificar instalación

- Abrir Símbolo del sistema (CMD) o Terminal de VS Code
- Ejecutar: `git --version`
- Nos debería mostrar algo similar a: `git version 2.54.0.windows.1`

PARTE B: Configuración inicial de Git

Paso 04: Configurar nombre de Usuario

```
git config --global user.name "Tu Nombre"
```

Paso 05: Configurar correo electrónico

```
git config --global user.email "correo@ejemplo.com"
```

Importante: Estos datos se usarán para identificar quién hizo cada cambio en el proyecto.

Paso 06: Verificar configuración

```
git config --list
```

Debe mostrar:

- user.name
- user.email

PARTE C: Creación del Proyecto Web

Paso 07: Crear Carpeta del Proyecto

- En VSCode crear una carpeta llamada `guia07-mi-proyecto-web`
- Crear los 3 archivos básicos del proyecto:
- **index.html**

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Mi Proyecto Web</title>

    <!-- Enlace al archivo CSS -->
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Mi Primer Proyecto con Git</h1>

    <p>Ingrese su nombre:</p>

    <input type="text" id="nombre" placeholder="Escribe tu nombre" />

    <button onclick="saludar()">Saludar</button>

    <p id="resultado"></p>

    <!-- Enlace al archivo JavaScript -->
    <script src="script.js"></script>
  </body>
</html>
```



- **styles.css**

```
body {  
    font-family: Arial, sans-serif;  
    background-color: #f0f8ff;  
    text-align: center;  
    margin-top: 50px;  
}  
  
h1 {  
    color: #2c3e50;  
}  
  
input {  
    padding: 8px;  
    margin: 10px;  
}  
  
button {  
    padding: 8px 15px;  
    background-color: #3498db;  
    color: white;  
    border: none;  
    cursor: pointer;  
}  
  
button:hover {  
    background-color: #2980b9;  
}
```

- **script.js**

```
function saludar() {  
    // Obtener el valor del input  
    let nombre = document.getElementById("nombre").value;  
  
    // Validar si el usuario escribió algo  
    if (nombre === "") {  
        document.getElementById("resultado").innerText =  
            "Por favor, ingresa tu nombre.";  
    } else {  
        document.getElementById("resultado").innerText =  
            "Hola " + nombre + ", bienvenido al sistema.";  
    }  
}
```

En este punto el proyecto aún **NO** está controlado por Git

PARTE D: Inicializar Git

Paso 08: Inicializar Repositorio

- Ejecutar: `git init`
- Git creó una carpeta oculta llamada: `.git`
- Aquí se guardará todo el historial del proyecto

Paso 09: Ver estado del proyecto

- Ejecutar: `git status`
- Git mostrará archivos en rojo (no rastreados)
- Ejemplo:
Untracked files:
`index.html`
`styles.css`
`script.js`
- Significa que Git detecta los archivos, pero aún no los controla.



PARTE E: Primer Versionado del Proyecto

Paso 10: Agregar archivos al Area de Preparación

- Ejecutar: `git add .`
- Con esto indicamos a Git que *"Estos archivos deseamos guardarlos en la siguiente versión"*

Paso 11: Verificar estado

- Ejecutar: `git status`
- Archivos ahora aparecen en verde, listos para ser guardados

```
new file:   index.html
new file:   script.js
new file:   styles.css
```

Paso 12: Crear el Primer commit

- Ejecutar: `git commit -m "Versión inicial del proyecto web"`
- Se visualizará la inserción de los archivos:

```
3 files changed, 64 insertions(+)
create mode 100644 index.html
create mode 100644 script.js
create mode 100644 styles.css
```

PARTE F: Simulación de Cambios en el Sistema

Paso 13: Modificar un archivo

- Editar `index.html`:

```
<h1>Bienvenido a mi Página Web</h1>
```

Paso 14: Ver cambios

- Ejecutar: `git status`
- Git detecta que el archivo fue modificado

```
modified:   index.html
```

Paso 15: Registrar Nueva Versión

- Ejecutar los comandos:

```
git add .
git commit -m "Se actualiza el contenido principal"
```

Paso 16: Ver Historial de Versiones

- Ejecutar: `git log`
Git abre una vista tipo "visor" (llamado **pager**, normalmente `less`), donde puedes desplazarte. Para salir del listado, presionamos la tecla "q".
- Ejecutar: `git log --oneline`
Git muestra los commits en formato corto.



6.2. ACTIVIDAD 02: Publicación del Proyecto en GitHub y uso Básico de Ramas.

Objetivo: Publicar el proyecto web local en GitHub, sincronizar cambios y crear una rama para simular una mejora del sistema.

Contexto:

En la Actividad 01, el estudiante ya creó un proyecto web y registró cambios con Git de manera local.

Ahora el proyecto será llevado a GitHub para:

- Respalidar el código en la nube.
- Compartir el proyecto mediante un enlace.
- Preparar el trabajo colaborativo.
- Utilizar ramas para desarrollar mejoras sin afectar la versión principal.

PARTE A: Crear una cuenta en GitHub

Paso 01: Ingresar a GitHub

- Abrir el navegador e ingresar a: <https://github.com>
- Hacer clic en “Sign up”
- Completar los datos solicitados
- Confirmar la cuenta

Paso 02: Crear un Nuevo Repositorio

- En GitHub, hacer clic en el botón: *New*
Repository name: guia07-mi-proyecto-web
Description: Proyecto web básico usando HTML, CSS, JavaScript, Git y GitHub
Visibility: Public
- Clic en el botón *Create repository*.

Importante: No marcar estas opciones:

- Add a README file
- Add .gitignore
- Choose a license

¿Por qué?

Porque el proyecto ya existe en la computadora del estudiante. Si GitHub crea archivos adicionales, puede generar conflictos al conectar el repositorio local.

Resultado esperado:

GitHub mostrará una página con instrucciones para subir un repositorio existente. El estudiante debe ubicar una dirección similar a: <https://github.com/usuario/guia07-mi-proyecto-web.git>

PARTE B: Conectar el proyecto local con GitHub

Paso 03: Abrir el Proyecto en Visual Studio Code

- Ejecutar VS Code y abrir el proyecto creado en la Actividad 01
- Ejecutar el Terminal
- Verificar que estamos dentro del proyecto. Ejecutar: `pwd`
- Debe mostrar una ruta parecida a: `C:\Users\Usuario\Documents\guia07-mi-proyecto-web`

Paso 04: Verificar el Estado de Git

- Ejecutar: `git status`
Resultado esperado: *nothing to commit, working tree clean*
Esto significa que no hay cambios pendientes.



Paso 05: Verificar el nombre de la Rama Principal

- Ejecutar: `git branch`
- Puede aparecer: ** main o * master*
- Para esta guía se trabajará con *main*.
- Si aparece *master*, cambiar el nombre con: `git branch -M main`

Paso 06: Conectar el Repositorio Local con GitHub

- Copiar la URL del repositorio creado en GitHub
- Ejecutar el siguiente comando reemplazando la URL copiada:
`git remote add origin https://github.com/usuario/mi-proyecto-web.git`

Paso 07: Verificar la Conexión Remota

- Ejecutar: `git remote -v`
Resultado esperado:
origin <https://github.com/usuario/mi-proyecto-web.git> (fetch)
origin <https://github.com/usuario/mi-proyecto-web.git> (push)

Esto confirma que el proyecto local ya está conectado con GitHub.

PARTE C: Subir el Proyecto a GitHub

Paso 08: Enviar el Proyecto a GitHub

- Ejecutar: `git push -u origin main`
-

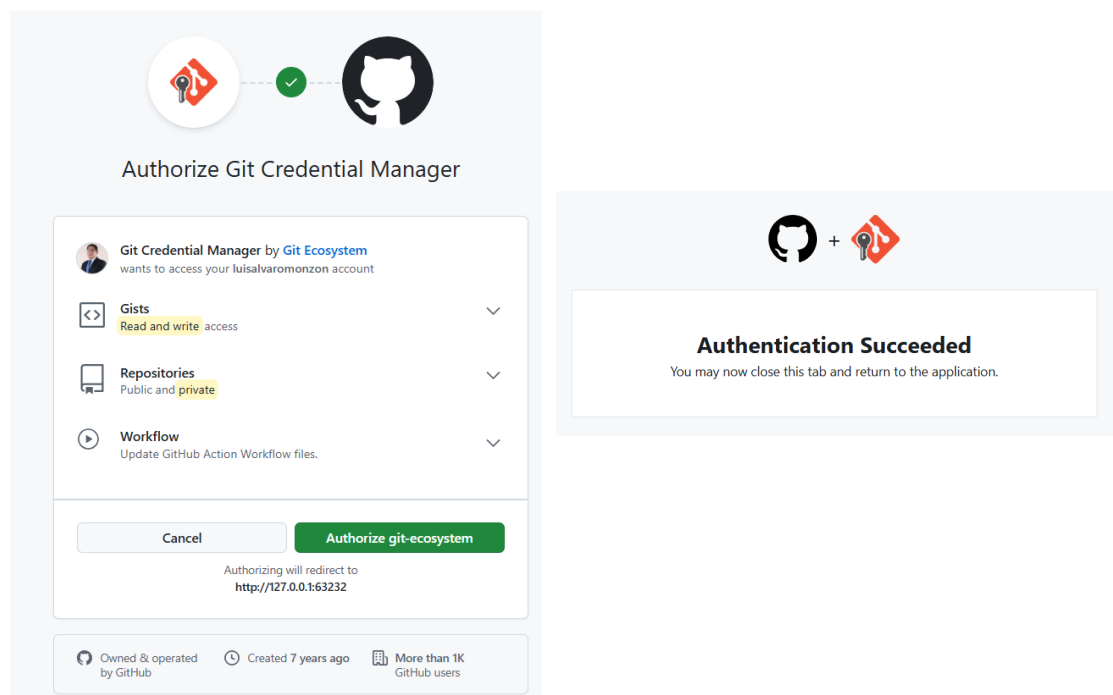
Paso 09: Iniciar Sesión si GitHub lo solicita

Puede aparecer una ventana para iniciar sesión en GitHub.

El estudiante debe:

- Iniciar sesión.
- Autorizar Git.
- Esperar que finalice el proceso.

Nota: GitHub ya no permite subir código usando contraseña normal en algunos casos. Si aparece una ventana de autorización, se debe aceptar desde el navegador.





Paso 10: Verificar en GitHub

- Volver al navegador y actualizar la página del repositorio.
- Debe aparecer:
index.html
styles.css
script.js
Historial de commits

Resultado esperado:

- El proyecto local ahora está publicado en GitHub.
- El estudiante podrá compartir su proyecto mediante la URL del repositorio.

PARTE D: Realizar un Cambio local y Actualizar GitHub

Paso 11: Modificar el Proyecto Local

- Abrir *index.html* y agregar debajo del título:
<p>Este proyecto fue publicado usando GitHub.</p>

Paso 12: Verificar cambios

- Ejecutar: `git status`
- Resultado esperado:
modified: index.html

Paso 13: Preparar cambios

- Ejecutar: `git add .`

Paso 14: Crear un Nuevo commit

- Ejecutar: `git commit -m "Se agrega mensaje de publicación en GitHub"`

Paso 15: Subir Cambios a GitHub

- Ejecutar: `git push`

Paso 16: Verificar Cambios en GitHub

- Actualizar el repositorio en el navegador.
- Ingresar al archivo *index.html*, debe verse el nuevo párrafo agregado.

PARTE E: Crear una Rama para una mejora del sistema

Paso 17: Crear una Nueva Rama

- Ejecutar: `git branch feature-formulario`
- Esta rama se usará para desarrollar una mejora sin afectar directamente la rama principal.

Paso 18: Cambiar a la Nueva Rama

- Ejecutar: `git checkout feature-formulario`

Resultado esperado:

Switched to branch 'feature-formulario'

Paso 19: Verificar Rama Actual

- Ejecutar: `git branch`

Resultado esperado:

```
* feature-formulario  
main
```



PARTE F: Implementar una mejora en la Rama

Paso 20: Modificar el archivo index.html

- Agregar un pequeño formulario debajo del contenido principal:

```
<h2>Formulario de contacto</h2>

<input type="email" id="correo" placeholder="Ingrese su correo">
<button onclick="validarCorreo()">Validar correo</button>

<p id="mensajeCorreo"></p>
```

Paso 21: Modificar el archivo script.js

- Agregar la siguiente función:

```
function validarCorreo() {
    let correo = document.getElementById("correo").value;

    if (correo === "") {
        document.getElementById("mensajeCorreo").innerText = "Debe ingresar un correo.";
    } else {
        document.getElementById("mensajeCorreo").innerText = "Correo registrado correctamente.";
    }
}
```

Paso 22: Probar el Proyecto

- Abrir index.html en el navegador y verificar que:
 - Aparece el formulario.
 - Permite ingresar un correo.
 - Muestra un mensaje al hacer clic.

Paso 23: Registrar Cambios en la Rama

- Ejecutar:

```
git add .
git commit -m "Se agrega formulario de contacto"
```

PARTE G: Fusionar la mejora con la Rama Principal

Paso 24: Volver a la Rama Principal

- Ejecutar: `git checkout main`

Paso 25: Fusionar la Rama feature-formulario

- Ejecutar: `git merge feature-formulario`
- Esto integra los cambios de la rama *feature-formulario* dentro de *main*.

Paso 26: Subir la versión final a GitHub

- Ejecutar: `git push`

Paso 27: Verificar en GitHub

- Actualizar el repositorio en GitHub y revisar que los archivos contienen la mejora realizada.



PARTE H: Revisar Historial del Proyecto

Paso 28: Ver Historial Resumido

- Ejecutar: `git log --oneline`
- Debe observarse algo similar a:
260a919 (HEAD -> main, origin/main, feature-formulario) Se agrega formulario de contacto
0874498 Se agrega mensaje de publicación en GitHub
0b48f4d Se actualiza el contenido principal
b7e477b Versión inicial del proyecto web

Paso 29: Ver Ramas Creadas

- Ejecutar: `git branch`
- Debe aparecer:
feature-formulario
* main

7. EJERCICIOS PROPUESTOS.

7.1. EJERCICIO PROPUESTO 01: Evolución Controlada de un Sistema Web

Objetivo:

Aplicar control de versiones utilizando Git y GitHub, registrando la evolución progresiva de un sistema web.

Enunciado del ejercicio

A partir del proyecto desarrollado en la guía, el estudiante deberá construir y mejorar progresivamente su sistema web, registrando cada cambio mediante commits y publicando el resultado en GitHub. El desarrollo debe evidenciar claramente la evolución del sistema desde una versión inicial hasta una versión mejorada.

Requerimientos del Desarrollo:

1. Proyecto ase

El sistema debe partir de un proyecto web funcional que incluya:

- Estructura HTML
- Estilos CSS
- Funcionalidad básica con JavaScript

2. Evolución progresiva del sistema

El estudiante debe desarrollar el proyecto en múltiples versiones, incorporando mejoras en cada etapa.

Debe incluir como mínimo:

Versión 1: Estructura inicial (ya desarrollada en la guía)

- Página HTML funcional
- Contenido básico

Versión 2: Estilos del sistema

- Aplicación de estilos CSS
- Mejora visual del contenido

Versión 3: Funcionalidad

- Uso de JavaScript (evento, validación o interacción)

Versión 4: Mejora del sistema (investigación)



El estudiante debe implementar una mejora adicional, por ejemplo:

- Validación más completa
- Mejora en la interfaz
- Interacción adicional
- Mensajes dinámicos

3. Uso de Git (control de versiones)

El desarrollo debe evidenciar:

- Registro de cambios mediante commits
- Mínimo 4 commits
- Mensajes claros y descriptivos
- Evolución lógica del sistema

4. Uso de GitHub

El proyecto debe:

- Estar publicado en GitHub
- Contener todo el historial de commits
- Reflejar la evolución del sistema en la plataforma

5. Historial del Proyecto

El repositorio debe mostrar:

- Secuencia ordenada de cambios.
- Evidencia clara de mejora en cada versión.
- Coherencia entre commits y funcionalidades.

Entregable:

El estudiante deberá presentar:

- Enlace del repositorio en GitHub
- Proyecto funcional
- Historial de commits visible

7.2. EJERCICIO PROPUESTO 02: Desarrollo de una Agenda de Contactos con Git y GitHub

Objetivo:

Desarrollar una aplicación web tipo Agenda de Contactos utilizando HTML, CSS y JavaScript, aplicando control de versiones con Git y GitHub y organizando el desarrollo mediante funcionalidades.

Enunciado del ejercicio:

El estudiante deberá desarrollar desde cero una aplicación web que permita gestionar contactos de manera interactiva. El desarrollo deberá realizarse aplicando control de versiones con Git y utilizando GitHub como repositorio remoto, organizando el trabajo mediante funcionalidades independientes.

Requerimientos del Sistema:

1. Funcionalidades obligatorias

El sistema debe permitir:

- Registrar un nuevo contacto
- Mostrar la lista de contactos
- Eliminar contactos
- Visualizar la información de cada contacto

2. Estructura de los datos

Cada contacto debe contener como mínimo:

- Nombre
- Número de teléfono
- Correo electrónico



3. Interacción con el usuario

El sistema debe incluir:

- Uso de eventos (click, input, submit)
- Manipulación dinámica del DOM
- Actualización inmediata de la lista de contactos
- Mensajes de validación o confirmación

4. Diseño del Sistema

La aplicación debe:

- Tener una interfaz ordenada y clara
- Utilizar estilos CSS
- Diferenciar visualmente los contactos (tarjetas, lista, etc.)

5. Uso de Git (Control de Versiones)

El desarrollo debe evidenciar:

- Inicialización del repositorio
- Registro de cambios mediante commits
- Mínimo 6 commits
- Mensajes claros y descriptivos
- Evolución progresiva del sistema

6. Uso de GitHub

El proyecto debe:

- Estar publicado en GitHub
- Mantener sincronización entre repositorio local y remoto
- Mostrar el historial completo de desarrollo

7. Desarrollo mediante funcionalidades (ramas)

El estudiante debe organizar el desarrollo mediante:

- Mínimo 3 ramas (features)
- Cada rama debe representar una funcionalidad del sistema

Ejemplo sugerido:

feature-agregar-contacto
feature-eliminar-contacto
feature-listar-contactos

8. Integración del sistema:

El sistema final debe:

- Integrar todas las funcionalidades en la rama principal
- Funcionar correctamente en el navegador
- No presentar errores visibles

Entregable:

El estudiante deberá presentar:

- Enlace del repositorio en GitHub
- Proyecto funcional
- Historial de commits visible
- Evidencia del uso de ramas



8. PROCESO DE EVALUACIÓN:

Criterios de evaluación de la Guía:

Criterios de evaluación	Descripción	Puntaje por Ítem
1. Actividad 01	Desarrollo de la Actividad 01 de la Guía	05
2. Actividad 02	Desarrollo de la Actividad 02 de la Guía	05
3. Ejercicio Propuesto 01	Desarrollo del Ejercicio Propuesto 01	05
4. Ejercicio Propuesto 02	Desarrollo del Ejercicio Propuesto 02	05
TOTAL		20